



Universidade do Minho
Escola de Engenharia

12/10/2025

Universidade do Minho - Escola de Engenharia

Comunicações por Computador - TP1

Gonçalo Castro
a107337

Rita Cunha
a104261

Pedro Figueiredo
a104354

Índice

1 Parte I: Instalação, configuração e validação da rede de testes	4
1.1 Criação de uma topologia de testes	4
1.2 Testes elementares de conectividade	4
1.3 Testes TCP e UDP em condições adversas	7
2 Parte II: Utilização de serviços de transferência de ficheiros	7
2.1 Transfira os dois ficheiros <i>file1</i> e <i>file2</i> a partir do PC3 (descarga do PC1 para o PC3) via TFTP e FTP.	7
2.2 Transfira os dois ficheiros <i>file1</i> e <i>file2</i> a partir do PC4 (descarga do PC1 para o PC4) via TFTP, FTP e HTTP.	9
2.3 Transfira os dois ficheiros <i>file1</i> e <i>file2</i> a partir do PC2 (descarga do PC1 para o PC2) via TFTP, FTP e HTTP.	10
2.4 Prepare e execute uma captura de tráfego de uma transferência TCP realizada, por exemplo, com iperf3 ou curl.	11
3 Parte III: Uso da camada de transporte por parte das aplicações	12
3.1 Com base no trabalho realizado, identifique para cada aplicação executada, qual o protocolo de aplicação, o protocolo de transporte, porta de atendimento e overhead de transporte.	12
4 Conclusões	13

Lista de Figuras

Figura 1	Topologia	4
Figura 2	Execução dos comandos traceroute e ping do PC1 → PC2	5
Figura 3	Execução dos comandos traceroute e ping do PC1 → PC3	5
Figura 4	Execução dos comandos traceroute e ping do PC1 → PC4	5
Figura 5	Execução dos comandos traceroute e ping do PC2 → PC3	5
Figura 6	Execução dos comandos traceroute e ping do PC2 → PC4	5
Figura 7	Execução dos comandos traceroute e ping do PC3 → PC4	5
Figura 8	iperf com PC1 como server	6
Figura 9	iperf com PC2 como server	6
Figura 10	iperf com PC3 como server	6
Figura 11	iperf com PC4 como server	6
Figura 12	Tabela de resultados	6
Figura 13	Testes TCP e UDP entre PC2 e PC4 com o iperf3.	7
Figura 14	Output do wireshark para transferência do file1 por FTP	8
Figura 15	Diagrama de Transferência por FTP do file1 do PC1 para o PC3	8
Figura 16	Diagrama de Transferência por TFTP do file1 do PC1 para o PC3	9
Figura 17	Output do Wireshark para transferência do file1 por HTTP	10
Figura 18	Output do Wireshark para transferência do file1 por HTTP	10
Figura 19	Captura de tráfego TCP com retransmissão de segmentos	11
Figura 20	Captura de tráfego TCP com atualização de valores de janela de recepção	11
Figura 21	Tabela dos protocolos aplicação e de transporte, portas de atendimento e overhead de transporte	12

Resumo

Neste trabalho foi construída e analisada uma topologia de rede em anel composta por quatro *routers* e quatro *hosts*, com diferentes parâmetros de atraso, débito e perdas, utilizando o **simulador CORE**.

Após a configuração e validação da rede, realizaram-se testes de conectividade e desempenho com ferramentas como *ping*, *traceroute* e *iperf3*, verificando-se a coerência entre os resultados e as condições impostas nas conexões.

Foram ainda conduzidos testes de transferência de ficheiros através dos protocolos **FTP**, **TFTP** e **HTTP**, analisando-se o comportamento de cada um com recurso ao **Wireshark**. Observou-se o estabelecimento e encerramento das conexões TCP, a sequência de segmentos trocados, o controlo de fluxo e as retransmissões devido às perdas de pacotes.

Compararam-se também as transferências sob **TCP** e **UDP**, identificando as diferenças entre fiabilidade, débito e taxa de perdas. Por fim, foi estudado o papel da camada de transporte em diversas aplicações, relacionando protocolos, portas e *overheads*.

1 Parte I: Instalação, configuração e validação da rede de testes

1.1 Criação de uma topologia de testes

Usando a interface gráfica do CORE no modo de edição, construa uma topologia com quatro roteadores. Faça uma ligação do nó n1 para o nó n2, do nó 2 para o nó 3, do nó 3 para o nó 4 e finalmente do nó 4 para o nó 1, resultando numa topologia em anel(...)

Configure as demais ligações, entre os roteadores, da seguinte maneira:

- Entre os nós 1 e 2: Utilize um débito de 1 Mbps, atraso de 20 ms, perdas de 10% e 10% de duplicações.
- Entre os nós 2 e 3: Utilize um débito de 2 Mbps, atraso de 10 ms e perdas de 5%.
- Entre os nós 3 e 4: Utilize um débito de 5 Mbps, atraso de 5 ms e perdas de 1%.
- Entre os nós 4 e 1: Utilize um débito de 10 Mbps, atraso de 0 ms e perdas de 0%.

a) Execute a simulação, e faça os testes necessários para poder responder a todas questões que se seguem. Inclua no relatório uma imagem da topologia criada e em execução no CORE.

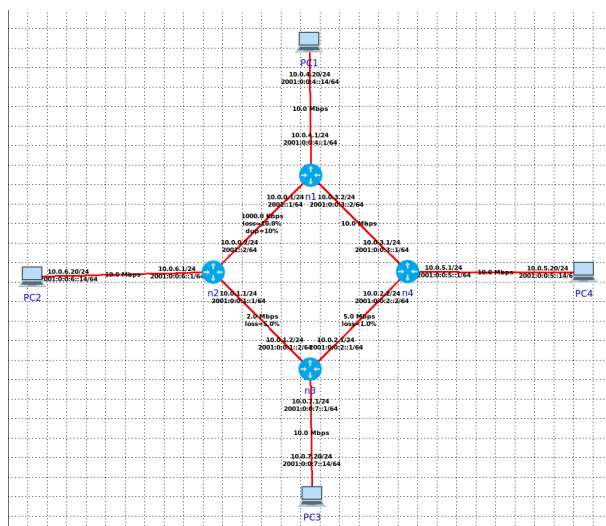


Figura 1: Topologia

1.2 Testes elementares de conectividade

Verifique que todas as rotas foram configuradas com sucesso e demonstre que os hosts possuem ligação entre si. Utilize as ferramentas *traceroute*, *ping* e *iperf* para verificar as rotas entre hosts, as estimativas de perdas de pacotes, atrasos e débito fim-a-fim.

a) Preencha uma pequena tabela com os valores obtidos.

Foram realizados testes de conectividade entre todos os hosts da rede através dos comandos *ping*, *traceroute* e *iperf3*. Os testes permitiram observar o caminho percorrido pelos pacotes, as perdas, o atraso médio e o débito fim-a-fim, confirmando a correta configuração das rotas e a coerência com os parâmetros definidos na topologia.

Começando pelo host PC1, conseguimos verificar que este tem comunicação com os restantes hosts através do recurso ao *ping*. Na ligação para o PC2, registou-se uma perda de cerca de 14% e um RTT

médio de 37.411 ms, enquanto para o PC3 a perda foi ligeiramente superior, com 15% e um RTT semelhante. Já na comunicação com o PC4, não se observaram perdas de pacotes e o RTT foi muito reduzido (0.626 ms), devido às melhores condições do enlace utilizado.

```

PC1->PC2                                     root@PC1: /volume (on a001834f5b01)
root@PC1:/volume# traceroute 10.0.6.20
traceroute to 10.0.6.20 (10.0.6.20), 30 hops max, 60 byte packets
 1 10.0.4.1 (10.0.4.1) 0.227 ms 0.260 ms 0.230 ms
 2 * 10.0.0.2 (10.0.0.2) 42.336 ms 43.024 ms
 3 10.0.6.20 (10.0.6.20) 43.797 ms * *
root@PC1:/volume# ping -c 100 -q 10.0.7.20
PING 10.0.7.20 (10.0.7.20) 56(84) bytes of data:
--- 10.0.7.20 ping statistics ---
100 packets transmitted, 86 received, +6 duplicates, 14% packet loss, time 99523ms
rtt min/avg/max/ndev = 36.995/37.411/38.397/0.245 ms
root@PC1:/volume#

```

Figura 2: Execução dos comandos traceroute e ping do PC1 → PC2

```

PC1->PC3                                     root@PC1: /volume (on a001834f5b01)
root@PC1:/volume# traceroute 10.0.7.20
traceroute to 10.0.7.20 (10.0.7.20), 30 hops max, 60 byte packets
 1 10.0.4.1 (10.0.4.1) 0.250 ms 0.294 ms 0.334 ms
 2 * 10.0.0.2 (10.0.0.2) 41.991 ms 42.533 ms
 3 10.0.2.1 (10.0.2.1) 37.797 ms 38.033 ms *
 4 * * *
 5 * * *
 6 * * *
 7 * 10.0.7.20 (10.0.7.20) 36.915 ms 37.146 ms
root@PC1:/volume# ping -c 100 -q 10.0.7.20
PING 10.0.7.20 (10.0.7.20) 56(84) bytes of data:
--- 10.0.7.20 ping statistics ---
100 packets transmitted, 85 received, +7 duplicates, 15% packet loss, time 99653ms
rtt min/avg/max/ndev = 36.868/37.385/37.955/0.228 ms
root@PC1:/volume#

```

Figura 3: Execução dos comandos traceroute e ping do PC1 → PC3

```

PC1->PC4                                     root@PC1: /volume (on a001834f5b01)
root@PC1:/volume# traceroute 10.0.5.20
traceroute to 10.0.5.20 (10.0.5.20), 30 hops max, 60 byte packets
 1 10.0.4.1 (10.0.4.1) 0.241 ms 0.266 ms 0.326 ms
 2 10.0.3.1 (10.0.3.1) 0.366 ms 0.407 ms 0.414 ms
 3 10.0.5.20 (10.0.5.20) 0.523 ms 0.569 ms 0.624 ms
root@PC1:/volume# ping -c 100 -q 10.0.5.20
PING 10.0.5.20 (10.0.5.20) 56(84) bytes of data:
--- 10.0.5.20 ping statistics ---
100 packets transmitted, 100 received, 0% packet loss, time 101398ms
rtt min/avg/max/ndev = 0.524/0.626/0.749/0.036 ms
root@PC1:/volume#

```

Figura 4: Execução dos comandos traceroute e ping do PC1 → PC4

Passando a observar agora o host PC2, através do comando ping verifica-se que este tem ligação com o PC3 e o PC4, não sendo necessário testar novamente a ligação com o PC1, já comprovada anteriormente. Detetou-se uma perda de pacotes de cerca de 11% para o PC3 e de 19% para o PC4.

```

PC2->PC3                                     root@PC2: /volume (on a001834f5b01)
root@PC2:/volume# traceroute 10.0.7.20
traceroute to 10.0.7.20 (10.0.7.20), 30 hops max, 60 byte packets
 1 10.0.6.1 (10.0.6.1) 0.245 ms 0.267 ms 0.322 ms
 2 10.0.1.2 (10.0.1.2) 21.235 ms 21.359 ms *
 3 10.0.7.20 (10.0.7.20) 21.716 ms 22.090 ms *
root@PC2:/volume# ping -c 100 -q 10.0.7.20
PING 10.0.7.20 (10.0.7.20) 56(84) bytes of data:
--- 10.0.7.20 ping statistics ---
100 packets transmitted, 89 received, 11% packet loss, time 99397ms
rtt min/avg/max/ndev = 21.217/21.541/21.987/0.152 ms
root@PC2:/volume#

```

Figura 5: Execução dos comandos traceroute e ping do PC2 → PC3

```

PC2->PC4                                     root@PC2: /volume (on a001834f5b01)
root@PC2:/volume# traceroute 10.0.5.20
traceroute to 10.0.5.20 (10.0.5.20), 30 hops max, 60 byte packets
 1 10.0.6.1 (10.0.6.1) 0.226 ms 0.273 ms 0.342 ms
 2 10.0.1.2 (10.0.1.2) 21.059 ms 21.439 ms 21.754 ms
 3 10.0.3.1 (10.0.3.1) 37.454 ms * *
 4 * * *
 5 * * *
 6 * * *
 7 * 10.0.5.20 (10.0.5.20) 36.793 ms 38.175 ms
root@PC2:/volume# ping -c 100 -q 10.0.5.20
PING 10.0.5.20 (10.0.5.20) 56(84) bytes of data:
--- 10.0.5.20 ping statistics ---
100 packets transmitted, 81 received, +7 duplicates, 19% packet loss, time 93747ms
rtt min/avg/max/ndev = 36.872/37.330/38.140/0.262 ms
root@PC2:/volume#

```

Figura 6: Execução dos comandos traceroute e ping do PC2 → PC4

Finalmente, analisou-se o host PC3, sendo apenas necessário verificar a ligação com o PC4, já que as restantes comunicações foram testadas nos passos anteriores. O comando ping confirmou a conectividade entre ambos, apresentando uma perda de cerca de 4%.

```

PC3->PC4                                     root@PC3: /volume (on a001834f5b01)
root@PC3:/volume# traceroute 10.0.5.20
traceroute to 10.0.5.20 (10.0.5.20), 30 hops max, 60 byte packets
 1 10.0.7.1 (10.0.7.1) 0.241 ms 0.270 ms 0.323 ms
 2 10.0.2.2 (10.0.2.2) 10.672 ms 10.768 ms 10.867 ms
 3 10.0.5.20 (10.0.5.20) 11.010 ms 11.131 ms *
root@PC3:/volume# ping -c 100 -q 10.0.5.20
PING 10.0.5.20 (10.0.5.20) 56(84) bytes of data:
--- 10.0.5.20 ping statistics ---
100 packets transmitted, 96 received, 4% packet loss, time 99276ms
rtt min/avg/max/ndev = 10.704/10.977/11.250/0.140 ms
root@PC3:/volume#

```

Figura 7: Execução dos comandos traceroute e ping do PC3 → PC4

Foram realizados testes de desempenho utilizando o comando `iperf3`, com o objetivo de determinar o débito fim-a-fim entre os diferentes hosts da topologia. Observou-se que a ligação PC1→PC2 apresentou um débito médio de aproximadamente 0,464 Mbit/s. Na ligação PC1→PC3, o débito medido foi de cerca de 0,383 Mbit/s, enquanto a ligação PC1→PC4 atingiu 9,55 Mbit/s. Relativamente às restantes medições, verificou-se um débito de aproximadamente 0,975 Mbit/s entre PC2→PC3, 0,551 Mbit/s entre PC2→PC4 e 3,45 Mbit/s entre PC3→PC4.

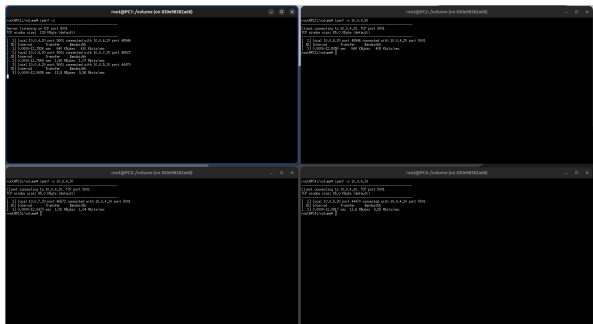


Figura 8: iperf com PC1 como server

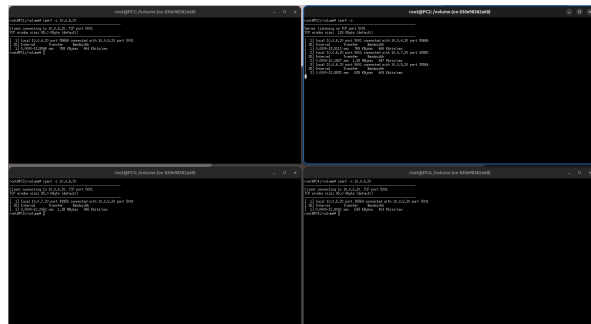


Figura 9: iperf com PC2 como server

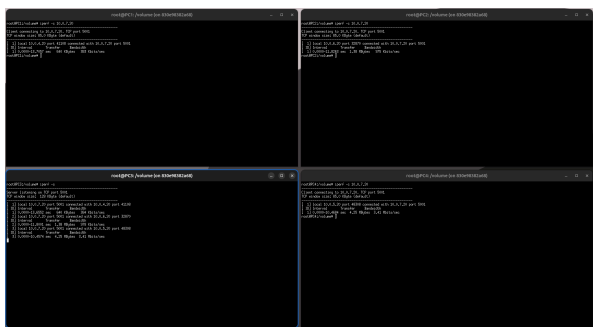


Figura 10: iperf com PC3 como server

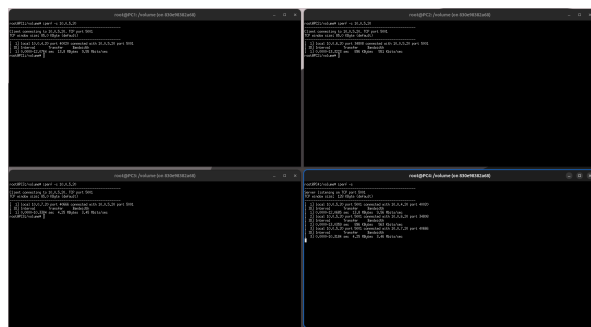


Figura 11: iperf com PC4 como server

Esta análise pode ser sintetizada na tabela seguinte, que apresenta de forma resumida os principais resultados obtidos nos testes realizados entre os diferentes hosts.

Origem	Destino	Hops	Perda de pacotes (%)	Atraso (ms)	Débito Fim-a-Fim (Mbit/s)
PC1	PC2	3	14	37.411	0.464
PC1	PC3	7	15	37.385	0.383
PC1	PC4	3	0	0.626	9.55
PC2	PC3	3	11	21.541	0.975
PC2	PC4	7	19	37.330	0.551
PC3	PC4	3	4	10.977	3.45

Figura 12: Tabela de resultados

1.3 Testes TCP e UDP em condições adversas

Utilizando o comando `iperf`, versão `iperf3`, realize testes de débito entre dois nós, escolhendo a ligação que apresenta mais perdas de pacotes. Sugestão: Inicie o servidor num dos nós (`iperf3 -s`) e depois o cliente no outro nó, primeiro gerando tráfego TCP (`iperf3 -c -t 20`) e depois tráfego UDP (`iperf3 -c -u -b 20M -t 20`)(...)

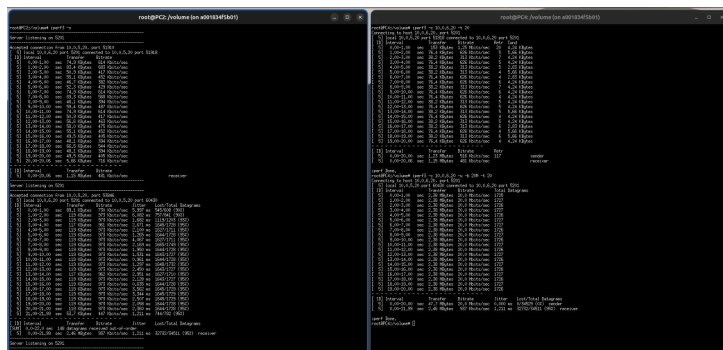


Figura 13: Testes TCP e UDP entre PC2 e PC4 com o iperf3.

a) Compare os resultados obtidos (débito, perdas, estabilidade).

Nos testes TCP, o débito médio obtido foi de cerca de 0,48 Mbit/s, com desempenho relativamente estável e poucas retransmissões. No caso do UDP, observou-se uma taxa de perdas bastante elevada (95%), com um débito médio próximo de 0,39 Mbit/s no recetor, apesar de o emissor ter mantido a taxa configurada de 20 Mbit/s. Estes resultados evidenciam o comportamento distinto dos dois protocolos: o TCP ajusta a taxa de envio à capacidade da rede, enquanto o UDP continua a enviar pacotes sem controlo de congestionamento, o que resulta em perdas significativas quando o canal se encontra saturado.

b) Com base nas diferenças observadas, discuta as vantagens e desvantagens de cada protocolo de transporte, dando exemplos de tipos de aplicações que beneficiem mais de um ou do outro.

O TCP apresenta maior fiabilidade e estabilidade, assegurando a entrega e ordenação dos pacotes e adaptando-se automaticamente às condições da rede. Por outro lado, o UDP é mais leve e rápido, mas não implementa mecanismos de controlo de fluxo ou retransmissão, tornando-o mais suscetível a perdas.

Assim, o TCP é mais indicado para aplicações que exigem fiabilidade, como transferência de ficheiros, navegação web ou email, enquanto o UDP é preferível em aplicações em tempo real, como streaming, jogos online ou chamadas VoIP, onde pequenas perdas são toleráveis em troca de menor latência.

2 Parte II: Utilização de serviços de transferência de ficheiros

2.1 Transfira os dois ficheiros *file1* e *file2* a partir do PC3 (descarga do PC1 para o PC3) via TFTP e FTP.

a) Identifique qual a rota que foi usada na transferência dos dados.

A rota usada na transferência pretendida é a mesma evidenciada pelo comando `traceroute` no Figura 3, dado que o PC1(servidor) possui o ficheiro que é encaminhado para o PC3(cliente); ou seja:

PC1(10.0.4.20) -> n1(10.0.4.1) -> n2(10.0.0.2) -> n3(10.0.1.2) -> PC3(10.0.7.20).

b) Apresente um diagrama temporal para a transferência do file1 por FTP. (...) Identifique no diagrama as fases de início de conexão, transferência de dados e fim de conexão, caso existam.

De seguida apresenta-se a captura de tráfego desta transferência, através do Wireshark no PC1:

[illegible]

Figura 14: Output do wireshark para transferência do file1 por FTP

O diagrama mostra a transferência do ficheiro *file1* entre o cliente (PC3) e o servidor (PC1) por **FTP**. Em primeiro lugar surge o *three-way handshake*, onde ambos estabelecem a ligação TCP. Segue-se a fase da transferência propriamente dita, em que o servidor envia blocos de dados de **1448 bytes** (e um último de 104 bytes), ao passo que o cliente responde com ACKs acumulativos e ajusta a sua janela de receção, refletindo o controlo de fluxo. Durante esta fase surgem também retransmissões do servidor, evidenciadas a vermelho, resultado de perdas que o TCP compensa para garantir fiabilidade. Por fim, o encerramento da ligação ocorre com a troca ordenada de segmentos FIN e ACK, completando o ciclo da comunicação.



Figura 15: Diagrama de Transferência por FTP do file1 do PC1 para o PC3

c) Apresente um diagrama temporal para a transferência do file1 por TFTP. Identifique, se aplicável, as fases de início de conexão, transferência de dados e fim de conexão. Identifique

também os tipos de segmentos trocados e os números de sequência usados quer nos dados como nas confirmações.

O diagrama representa o processo de transferência de um ficheiro através do **protocolo TFTP** entre o cliente (PC3: 10.0.7.20) e o servidor (PC1: 10.0.4.20). O processo inicia-se com o envio, pelo cliente, de um *Read Request* (RRQ) a solicitar o ficheiro *file1*. O servidor responde com o bloco de dados 1, ao qual o cliente envia o respetivo ACK (*acknowledgment*). A transferência decorre então de forma sequencial, com o servidor a enviar blocos de dados numerados (DATA) e o cliente a confirmar a receção de cada bloco com um ACK correspondente. Este ciclo DATA/ACK repete-se para todos os blocos, do 1 até ao 21, sendo que o último bloco (21) contém menos dados, sinalizando o final da transferência. O TFTP garante a fiabilidade através desta lógica simples: cada bloco só é considerado entregue quando o servidor recebe o ACK correspondente. Assim, mesmo sendo um protocolo sobre UDP, sem mecanismos nativos de fiabilidade, consegue assegurar a entrega correta do ficheiro.

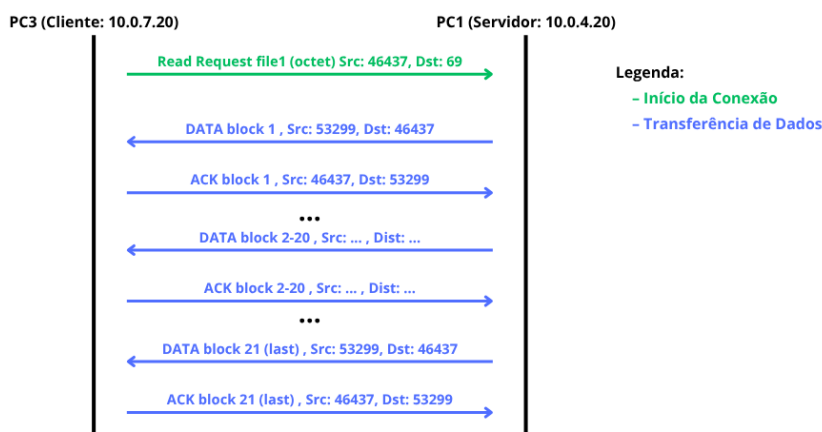


Figura 16: Diagrama de Transferência por TFTP do file1 do PC1 para o PC3

d) De que forma as perdas de pacotes afetaram o desempenho das aplicações nos dois casos anteriores? Qual a camada que lidou com as perdas: transporte ou aplicação? (...)

No caso do FTP, como este protocolo assenta sobre o TCP, as perdas de pacotes foram detetadas e corrigidas diretamente na camada de transporte, através de mecanismos automáticos de retransmissão, controlo de erros e ordenação de segmentos. Assim, a aplicação não foi afetada em termos de integridade do ficheiro, apenas se verificou um aumento no tempo total de transferência devido às retransmissões necessárias.

No caso do TFTP, que utiliza o UDP (sem mecanismos de fiabilidade), foi a própria camada de aplicação que teve de lidar com as perdas. O protocolo TFTP implementa retransmissões sempre que um ACK não é recebido dentro do tempo esperado, garantindo assim a entrega de todos os blocos de dados. Também aqui o impacto foi sobretudo ao nível do desempenho, com maior demora na transferência, mas sem comprometer a correção do ficheiro transferido.

2.2 Transfira os dois ficheiros *file1* e *file2* a partir do PC4 (descarga do PC1 para o PC4) via TFTP, FTP e HTTP.

a) Relativamente a esta transferência HTTP, identifique:

- O início e o fim da sessão TCP e os números de sequência e de confirmação (ACK) usados na conexão;
- O número de sequência inicial e a forma como é incrementado com cada pacote tanto pelo cliente quanto pelo servidor.

A sessão TCP inicia-se com o *three-way handshake*: **SYN** Seq=0 por parte do cliente, **SYN-ACK** Seq=0 Ack=1 por parte do servidor, **ACK** Seq=1 Ack=1 novamente por parte do cliente. Termina com a troca **FIN,ACK** / **ACK** (FIN-ACK Seq=130 pelo servidor, **ACK** final Seq=10462 feito pelo cliente). O cliente começa com o **GET** para sinalizar o seu pedido pelo *file1*. O servidor inicia o envio do mesmo em Seq=1 e incrementa de acordo com o tamanho dos segmentos (1448 bytes cada), ou seja **Seq= 1+1448, 2897(1449+1448), 4345(2897+1448), 5793(4345+1448), ...**, confirmados pelo cliente com ACKs sucessivos até ao último byte.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
2	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
3	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
4	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
5	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
6	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
7	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
8	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
9	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
10	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
11	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
12	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
13	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
14	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
15	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
16	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
17	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
18	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
19	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
20	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
21	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
22	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
23	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
24	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
25	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
26	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0

Figura 17: Output do Wireshark para transferência do file1 por HTTP

b) Com base nas transferências efetuadas, qual dos protocolos aplicacionais experimentados (TFTP, FTP e HTTP) seria o mais adequado para a obtenção dos ficheiros pelo PC4 (descarga do PC1 para o PC4)? Justifique.

O protocolo mais adequado é o HTTP, pois usa TCP para garantir fiabilidade, é simples (porta única), eficiente na transferência em blocos grandes e fácil de usar. O FTP também é fiável mas mais complexo, e o TFTP é menos eficiente e mais sensível a perdas.

2.3 Transfira os dois ficheiros *file1* e *file2* a partir do PC2 (descarga do PC1 para o PC2) via TFTP, FTP e HTTP.

a) Relativamente a esta transferência HTTP:

- **Identifique a perda e a duplicação de pacotes numa sessão TCP;**
- **Explique o impacto da perda e duplicação de pacotes numa sessão TCP, bem como os mecanismos usados pelo TCP**

para lidar com estas situações.

Na captura, a perda é visível no pacote 42 (TCP Retransmission Seq=0) e nos segmentos 58–72 (Out-Of-Order), onde o servidor reenviou dados não reconhecidos. A duplicação surge nos quadros 56 e 74, com **Dup ACKs** enviados pelo cliente para indicar falta de segmentos. O impacto é um aumento do tempo de transferência, pois os dados têm de ser reenviados, mas sem comprometer a integridade da sessão. O TCP resolve estas situações com **ACKs** acumulativos e duplicados, retransmissão rápida, *timeouts* e controlo de congestionamento, garantindo a entrega correta e ordenada ao destino.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
2	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
3	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
4	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
5	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
6	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
7	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
8	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
9	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
10	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
11	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
12	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
13	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
14	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
15	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
16	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
17	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
18	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
19	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
20	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
21	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
22	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
23	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
24	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
25	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0
26	0.000000	10.0.0.20	10.0.0.20	TCP	60	65535 → 65535 [RST] Seq=65535 Win=0 Len=0

Figura 18: Output do Wireshark para transferência do file1 por HTTP

b) Com base nas transferências efetuadas, qual dos protocolos aplicacionais experimentados (TFTP, FTP e HTTP) seria o mais adequado para a obtenção dos ficheiros pelo PC2 (descarga do PC1 para o PC2)? Justifique.

Com base nas evidências acima:

- TFTP mostrou-se problemático, com erros de blocos e *timeouts*, uma vez que usa UDP e depende de retransmissões na própria aplicação, tornando-o pouco fiável em redes com perdas.
- FTP funcionou corretamente e garantiu a integridade dos ficheiros, mas requer configuração extra (ex.: autenticação, modos de dados) e pode ser menos prático em termos de *firewall*/NAT.
- HTTP foi o mais simples e eficiente: a transferência decorreu sem erros, usando TCP para fiabilidade, e bastou um comando (wget) para descarregar os ficheiros de forma rápida.

Assim, o protocolo mais adequado para o PC2 obter ficheiros do PC1 é o HTTP, pela sua simplicidade, robustez e desempenho.

2.4 Prepare e execute uma captura de tráfego de uma transferência TCP realizada, por exemplo, com iperf3 ou curl.

Durante a transferência com iperf3 (sendo PC1 o servidor e PC3 o cliente) foi possível observar dois mecanismos fundamentais do TCP:

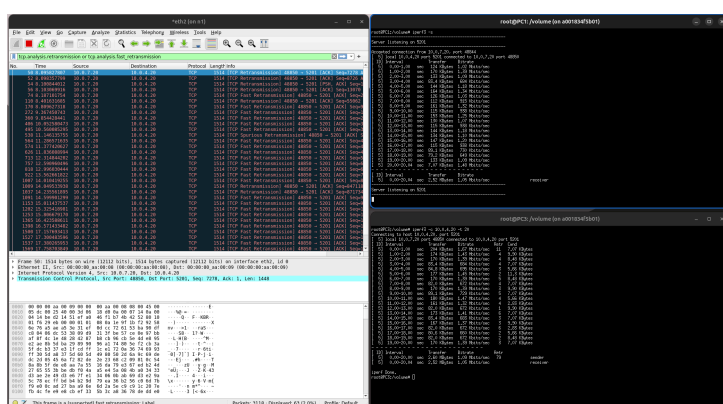


Figura 19: Captura de tráfego TCP com retransmissão de segmentos

• Retransmissão de segmentos:

Na captura da Figura 15, obtida com o filtro **tcp.analysis.retransmission or tcp.analysis.fast_retransmission**, são visíveis vários pacotes marcados como *TCP Retransmission* e *Fast Retransmission*. Isto acontece quando o emissor não recebe confirmações (ACKs) dentro do tempo esperado ou quando recebe ACKs duplicados, levando-o a reenviar os segmentos em falta. Este mecanismo garante a fiabilidade da comunicação, assegurando que todos os dados chegam corretamente ao destino.

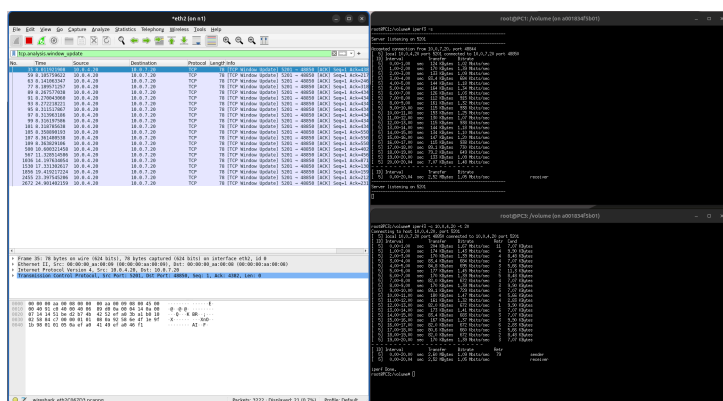


Figura 20: Captura de tráfego TCP com atualização de valores de janela de receção

• Controlo de Fluxo (janela de receção):

Evidenciado pela Anexo28, obtida com o filtro `tcp.analysis.window_update`, é possível observar o controle de fluxo do TCP através do campo *Window size value*. Por exemplo, no pacote nº 35 o recetor (10.0.4.20) envia **Ack=4382** com **Window=217**, indicando que apenas pode receber mais 217 bytes. Nos pacotes seguintes, o valor da janela aumenta (ex.: 870, 1450, 2320 bytes), à medida que o recetor liberta espaço no *buffer*. Este mecanismo garante que o emissor (10.0.7.20) nunca envia mais dados do que aqueles que o recetor consegue processar, evitando perdas por sobrecarga e assegurando uma transferência estável.

3 Parte III: Uso da camada de transporte por parte das aplicações

3.1 Com base no trabalho realizado, identifique para cada aplicação executada, qual o protocolo de aplicação, o protocolo de transporte, porta de atendimento e overhead de transporte.

Comando usado (aplicação)	Protocolo de Aplicação (se aplicável)	Protocolo de transporte (se aplicável)	Porta de atendimento (se aplicável)	Overhead de transporte em bytes (se aplicável)
wget, lynx ou via browser	HTTP	TCP	80	40
ssh, sftp	SSH	TCP	22	20
ftp	FTP	TCP	21	20
Tftp *				
telnet	Telnet	TCP	23	20
nslookup ou dig	DNS	UDP	53	8
Ping				
Traceroute		ICMP/UDP *	33434	8

Figura 21: Tabela dos protocolos aplicação e de transporte, portas de atendimento e overhead de transporte

A Figura 10 apresenta como diferentes comandos e aplicações utilizam a camada de transporte através de diversos protocolos. Por exemplo, ferramentas como wget, lynx ou navegadores utilizam o protocolo HTTP na camada de aplicação, que, por sua vez, opera sobre o TCP na camada de transporte, usando a porta 80 e com um overhead de 40 bytes.

O protocolo FTP, utilizado por comandos como ftp, também emprega o TCP, porém na porta 21, com um overhead de 20 bytes. Já o telnet, utilizado para acesso remoto, utiliza o protocolo Telnet na camada de aplicação e TCP na camada de transporte, com a porta 23.

Consultas DNS feitas com nslookup ou dig utilizam UDP na camada de transporte, geralmente na porta 53, oferecendo um overhead menor (8 bytes), o que reflete sua natureza leve e rápida, embora menos confiável que o TCP.

Por outro lado, o comando ping não utiliza protocolo de transporte (TCP/UDP), pois opera diretamente com o ICMP na camada de rede. O mesmo ocorre com o traceroute, que pode utilizar ICMP ou UDP (porta 33434), com overhead de apenas 8 bytes.

No caso do TFTP, o protocolo de aplicação deveria funcionar sobre UDP, pois é um protocolo simples e sem controle de ligação. No entanto, durante a captura foi observado que o comando tftp resultou em segmentos TCP marcados como Retransmission, indicando que a comunicação não foi estabelecida corretamente via UDP. Assim, apenas o protocolo de transporte (TCP) foi identificado na captura, sem ocorrência de tráfego real de TFTP, o que explica a ausência do protocolo de aplicação e a presença de retransmissões TCP.

Esses dados evidenciam que a escolha entre protocolos como TCP, UDP ou ICMP depende das exigências da aplicação: TCP garante confiabilidade com maior overhead, enquanto UDP é mais rápido e leve, mas menos confiável. ICMP, por sua vez, é usado para diagnósticos diretos na rede.

4 Conclusões

A análise prática confirmou o funcionamento esperado dos protocolos de rede e evidenciou as diferenças entre TCP e UDP. O TCP demonstrou-se mais fiável e robusto, assegurando a entrega ordenada e sem erros dos dados, embora com menor débito em condições adversas. Já o UDP, apesar da elevada taxa de perdas, apresentou menor latência e simplicidade, sendo adequado a aplicações em tempo real.

Nas transferências de ficheiros, o HTTP revelou-se o protocolo mais eficiente e fácil de utilizar, enquanto o FTP garantiu boa fiabilidade, mas com maior complexidade, e o TFTP mostrou limitações devido à ausência de controlo de erros nativo.

No geral, o trabalho permitiu consolidar os conceitos teóricos sobre a camada de transporte e perceber a importância de escolher o protocolo adequado para cada tipo de aplicação e condição de rede.